Introduction to Apache Spark

307401 — Big Data and Data Warehouses

Table of Contents

1. What is Apache Spark?
2. Spark vs Hadoop MapReduce
3. Spark Core Components
4. Spark Core Modules
5. Module Roadmap
6. Spark Architecture
7. Spark Driver
8. Spark Executor
9. Cluster Manager
10. Local Mode vs Cluster Mode
11. Languages Supported
12. Spark in Python (PySpark)
13. The Role of Py4J
14. Spark Data Objects
15. Spark SQL & DataFrames in Practice
16. Machine Learning with MLlib
17. Structured Streaming
18. Performance Optimization
19. Delta Lake & the Lakehouse
20. Summary & Next Steps

---

1. What is Apache Spark?

Apache Spark is a **distributed data processing engine** designed for large-scale analytics and machine learning.

- Started in **2009** at UC Berkeley's AMPLab as a research project; open-sourced in 2010
- Unlike Hadoop MapReduce (which writes intermediate results to disk after every step), Spark performs **in-memory computation** — drastically reducing disk I/O
- Uses a **Directed Acyclic Graph (DAG)** execution model instead of a rigid Map → Reduce pipeline
- Supports multiple high-level APIs: **RDDs, DataFrames, Datasets**
- Integrates with **SQL, Streaming, MLlib, and GraphX** within the same framework
- Runs standalone or on **YARN, Mesos, Kubernetes**; supports HDFS, S3, Cassandra, and more

---

Why is Spark faster than Hadoop MapReduce?

Reason	Detail
In-memory caching	Intermediate data stays in RAM
DAG execution model	Optimizes the full computation graph before running
Reduced disk writes	No HDFS write between pipeline stages
Rich APIs	Declarative operations allow automatic optimization

> Spark is widely adopted for ETL, analytics, real-time processing, and AI-driven pipelines.

---

2. Spark vs Hadoop MapReduce

```
┌──────────────────────────────────────────────────────────────────┐
│                HADOOP MAPREDUCE vs APACHE SPARK                  │
├──────────────────┬──────────────────────┬──────────────────────┤
│ Aspect           │ Hadoop MapReduce     │ Apache Spark         │
├──────────────────┼──────────────────────┼──────────────────────┤
│ Processing       │ Disk-based           │ In-memory            │
│                  │ Reads/writes HDFS    │ Keeps data in RAM    │
│                  │ after every step     │ between operations   │
├──────────────────┼──────────────────────┼──────────────────────┤
│ Speed            │ Baseline             │ Up to 100x faster    │
│                  │                      │ for iterative tasks  │
├──────────────────┼──────────────────────┼──────────────────────┤
│ Model            │ Rigid Map → Reduce   │ Flexible DAG         │
│                  │ two-stage only       │ with rich APIs       │
├──────────────────┼──────────────────────┼──────────────────────┤
│ Real-time        │ Not supported        │ Structured Streaming │
│                  │                      │ built-in             │
├──────────────────┼──────────────────────┼──────────────────────┤
│ Machine Learning │ Apache Mahout        │ MLlib — native,      │
│                  │ (limited, external)  │ scalable, pipelines  │
├──────────────────┼──────────────────────┼──────────────────────┤
│ Languages        │ Java primarily       │ Python, Scala,       │
│                  │                      │ Java, R, SQL         │
├──────────────────┼──────────────────────┼──────────────────────┤
│ Use Case         │ Batch ETL on large   │ ML, streaming,       │
│                  │ sequential datasets  │ interactive analytics │
└──────────────────┴──────────────────────┴──────────────────────┘
```

How Hadoop MapReduce Works (disk-heavy)

```
Input → [Map] → Write to HDFS → [Shuffle] → Write to HDFS → [Reduce] → Output
                    💾                            💾
         (disk write after every step — very slow for iterative jobs)
```

How Spark Works (memory-first)

```
Input → [Transform 1] → [Transform 2] → [Transform 3] → [Action] → Output
              ↕ RAM          ↕ RAM           ↕ RAM
         (data stays in memory — only writes to disk when you ask it to)
```

> **Important:** Spark does **not** replace HDFS. It uses HDFS as its storage layer, replacing only the MapReduce computation engine.

---

3. Spark Core Components

```
┌──────────────────────────────────────────────────────────────────────┐
│                        SPARK ECOSYSTEM                               │
├──────────────┬──────────────┬──────────────┬──────────────┬──────────┤
│  Structured  │    MLlib     │  Spark SQL   │  SparkR      │  GraphX  │
│  Streaming   │  (ML Library)│ (SQL API)    │  (R API)     │ (Graphs) │
│  Python/Scala│  Python/Scala│  SQL         │  R           │ Scala/Java│
├──────────────┴──────────────┴──────────────┴──────────────┴──────────┤
│                      DataFrame API                                   │
│              High-level API for structured data operations           │
│                      Python / Scala / Java                           │
├──────────────────────────────────────────────────────────────────────┤
│                    Spark Core  (RDD API)                             │
│         Memory management · Fault recovery · Task scheduling         │
│                      Python / Scala / Java                           │
└──────────────────────────────────────────────────────────────────────┘
```

- **Spark Core Engine** — foundation: manages memory, fault recovery, scheduling, and task distribution
- **High-level APIs** — DataFrame API, RDD API, SQL API — available in Python, Scala, Java, R
- **Specialized libraries** — Spark SQL, MLlib, GraphX, Structured Streaming extend Spark's capabilities

---

4. Spark Core Modules

1. Spark Core

- **RDDs** (Resilient Distributed Datasets): the fundamental data structure — immutable, fault-tolerant, distributed collections
- **Transformations** (e.g., `map`, `filter`) create a new RDD; **Actions** (e.g., `count`, `collect`) execute and return results

2. Spark SQL

- Query structured data using SQL or the DataFrame API
- Optimized by the **Catalyst Query Optimizer**
- Commonly used for analyzing large datasets

3. MLlib — Machine Learning Library

- Scalable algorithms: regression, classification, clustering, collaborative filtering
- Pipeline API for end-to-end ML workflows

4. Spark Streaming

- Enables real-time data processing for continuous, live data feeds
- Modern API: **Structured Streaming** (treats streams as unbounded tables)

5. GraphX

- Graph processing library for graph-parallel computations
- Useful for social network analysis, PageRank, connected components

---

5. Module Roadmap

#	Notebook	Key Concepts
1	**RDDs, Transformations & Actions**	`parallelize`, `map`, `filter`, `flatMap`, `reduce`, lazy evaluation, word count
2	**DataFrames & Spark SQL**	`select`, `groupBy`, joins, SQL queries, Parquet, window functions
3	**Machine Learning with MLlib**	Pipelines, Regression, Classification, Clustering, CrossValidator
4	**Structured Streaming**	`readStream`, windows, watermarking, output modes, real-time ETL
5	**Performance Optimization**	Caching, partitioning, broadcast joins, Catalyst, benchmarking

> All notebooks run on **Google Colab** — no local installation required. Open each notebook via the Colab badge at the top.

---

6. Spark Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                      CLIENT (Your Code)                      │
└──────────────────────────────────────────────────────────────┘
                              │
                              │ submits job
                              ▼
┌──────────────────────────────────────────────────────────────┐
│                     DRIVER PROGRAM                           │
│   SparkSession / SparkContext                                │
│   • Builds logical plan                                      │
│   • Optimizes via Catalyst                                   │
│   • Generates DAG of tasks                                   │
└──────────────────────────────────────────────────────────────┘
            │
            │ requests resources
            ▼
```

```
┌──────────────────────────┐
│    CLUSTER MANAGER       │
│  YARN / Mesos /          │
│  Kubernetes /            │
│  Spark Standalone        │
└──────────────────────────┘
        │
        │ allocates workers
        ▼

┌──────────────────────────────────────────────────────────┐
│                    WORKER NODES                          │
│  ┌──────────────────┐     ┌──────────────────┐          │
│  │    Executor      │     │    Executor      │          │
│  │  ┌────┐  ┌────┐  │     │  ┌────┐  ┌────┐  │          │
│  │  │Task│  │Task│  │     │  │Task│  │Task│  │          │
│  │  └────┘  └────┘  │     │  └────┘  └────┘  │          │
│  │  [Cache/Memory]  │     │  [Cache/Memory]  │          │
│  └──────────────────┘     └──────────────────┘          │
└──────────────────────────────────────────────────────────┘
```

Key Components

Component	Role
Driver	Brain — creates DAG, optimizes plan, schedules tasks
Cluster Manager	Resource allocator — provides CPU, RAM, containers
Executor	Worker — executes tasks, stores cached data
Task	Smallest unit of work — one partition, one operation
SparkSession	Entry point for all Spark APIs in PySpark

```python
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MyApp") \
    .getOrCreate()

print("App Name:", spark.sparkContext.appName)
print("Master:",   spark.sparkContext.master)
```

---

7. Spark Driver

The **Spark Driver** is the central coordinator of a Spark application. It runs the main program and orchestrates execution across the cluster.

Key Responsibilities

1. Creating SparkSession / SparkContext

- Initializes the Spark environment and entry point

2. Building the Logical Plan

- When you write `df.groupBy().count()`, the driver builds a logical execution plan
- The **Catalyst Optimizer** then optimizes this plan

3. Generating the DAG

```
User Code → Logical Plan → Optimized Plan → DAG → Stages → Tasks
                                (Catalyst)
```

4. Task Scheduling

- Communicates with the Cluster Manager to allocate resources and launch executors

5. Monitoring & Coordination

- Tracks task progress, retries failed tasks, provides metrics via the **Spark UI** (`localhost:4040`)

Driver vs Executor

```
DRIVER = Brain
  → Creates DAG
  → Optimizes plan
  → Schedules tasks
  → Collects results

EXECUTOR = Muscles
  → Executes tasks
  → Stores cached data
  → Returns results to Driver
```

```python
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("DriverExample").getOrCreate()

data = [("Alice", 25), ("Bob", 30), ("Carol", 22)]
df = spark.createDataFrame(data, ["name", "age"])

# Driver builds the plan; Executors run the filter
result = df.filter(df.age > 26)
result.show()
```

---

8. Spark Executor

A **Spark Executor** is a distributed worker process launched on each worker node.

Key Responsibilities

1. Task Execution

- Runs multiple **tasks** in parallel
- Tasks come from the DAG built by the Driver

2. Data Storage (Caching & Shuffling)

- Holds intermediate data in memory (when cached/persisted)
- Handles **shuffle operations** — exchanging data across nodes during `groupBy`, `join`

3. Communication with Driver

- Reports task status and sends computed results back
- If an executor fails, Spark reschedules tasks on another executor (**fault tolerance**)

Executor Lifecycle

```
Application Starts → Executors Launched → Tasks Run → Application Ends → Executors
Terminated
                                                ↕
                                    (dynamic allocation: scale up/down)
```

Configuring Executors in PySpark

```
spark = SparkSession.builder \
    .appName("ExecutorExample") \
    .config("spark.executor.instances", "4") \  # 4 executors
    .config("spark.executor.memory",    "2g") \  # 2 GB RAM each
    .config("spark.executor.cores",     "2") \   # 2 CPU cores each
    .getOrCreate()
```

---

9. Cluster Manager

The **Cluster Manager** is responsible for **resource allocation** across nodes. Spark relies on it for CPU, memory, and scheduling.

Responsibilities

1. **Resource Allocation** — decides how many executors to launch, how much memory/CPU each gets
2. **Executor Management** — starts, stops, and monitors executors on worker nodes
3. **Driver Communication** — grants resources; driver then schedules tasks onto executors

4. **Isolation** — fair resource sharing in multi-tenant environments
5. **Fault Recovery** — provisions new executors if one fails

Supported Cluster Managers

Cluster Manager	Description
Standalone	Built-in Spark cluster manager — simple, no extra setup
YARN	Hadoop's resource manager — most common in enterprise
Mesos	General-purpose cluster manager
Kubernetes	Modern container-based — increasingly popular
Local	Single machine — for development and testing

```
# Connect to YARN cluster
spark = SparkSession.builder \
    .appName("ClusterExample") \
    .master("yarn") \           # or "local", "mesos", "k8s://"
    .config("spark.executor.instances", "4") \
    .getOrCreate()

print("Cluster Manager:", spark.sparkContext.master)
```

> **Summary:** Driver plans → Executors execute → Cluster Manager provides the infrastructure.

---

10. Local Mode vs Cluster Mode

```
LOCAL MODE                          CLUSTER MODE
──────────────────────────────      ──────────────────────────────────────
Single machine (your laptop)        Many machines (cloud or on-prem)

 ┌──────────────────────┐            ┌─────────┐    ┌─────────┐
 │  Driver + Executors  │            │ Driver  │    │ Worker  │ Executor
 │  (simulated threads) │            └─────────┘    └─────────┘
 └──────────────────────┘                 │
                                          │          ┌─────────┐
                                     Cluster         │ Worker  │ Executor
  local[*]  → all CPU cores          Manager         └─────────┘
  local[4]  → 4 cores only           (YARN/K8s)
  local[1]  → single thread                          ┌─────────┐
                                                     │ Worker  │ Executor
                                                     └─────────┘

  Best for: learning, debugging     Best for: production, large data
```

```
# Local mode (development)
spark = SparkSession.builder \
```

```
    .master("local[*]") \     # use all available cores
    .appName("LocalApp") \
    .getOrCreate()

# Cluster mode (production) — master set externally via spark-submit
spark = SparkSession.builder \
    .appName("ProductionApp") \
    .getOrCreate()
```

---

11. Languages Supported by Apache Spark

Apache Spark is **polyglot** — use your language of choice:

Language	Notes
Scala	Spark's native language — richest and most efficient API
Java	Well-supported, more verbose than Scala
Python (PySpark)	Most popular for data scientists; leverages Python's ecosystem
R (SparkR / sparklyr)	DataFrame APIs for statisticians
SQL	Via Spark SQL — query structured and semi-structured data

> **PySpark dominates industry adoption** because of Python's widespread use in analytics, AI, and machine learning.

---

12. Spark in Python (PySpark)

PySpark is the Python API for Apache Spark — it gives Python developers access to Spark's full distributed computing power.

How PySpark Works Internally

```
Your Python Code
      │
      │  (Py4J bridge — socket connection)
      ▼
┌──────────────┐        ┌──────────────────────────┐
│  Python      │        │          JVM             │
│  Process     │◄──────►│   Spark Core (Scala)     │
│  (Driver)    │        │   SparkContext           │
└──────────────┘        └──────────────────────────┘
                                   │
                                   │
                            ┌──────┴──────┐
                            ▼             ▼
                      ┌──────────┐   ┌──────────┐
                      │  Worker  │   │  Worker  │
                      │ (Python  │   │ (Python  │
```

```
                        │  pipe)   │      │  pipe)   │
                        └──────────┘      └──────────┘
```

- Python commands are translated into JVM calls via **Py4J**
- Spark executes on the JVM cluster; results return to Python
- Data transformations happen in the JVM — only results come back to Python

---

13. The Role of Py4J

Py4J is a Python-Java bridge library enabling seamless communication between Python and the JVM.

How It Works — Step by Step

```
1. You write Python code
        │
        ▼
2. Py4J translates the call into a JVM operation
        │
        ▼
3. JVM (Spark engine) executes on the cluster
        │
        ▼
4. Results sent back to Python via Py4J
```

- Without Py4J, PySpark could not access Spark's JVM-based APIs
- When you call `df.show()`, Py4J translates this Python method into a JVM operation that executes in Spark's distributed environment

---

14. Spark Data Objects

Spark provides three data abstractions. Understanding their differences determines which API you reach for.

14.1 — RDD (Resilient Distributed Dataset)

The **original Spark abstraction** introduced in Spark 1.0.

> An RDD is an **immutable, partitioned collection of objects** that can be processed in parallel across a cluster.

Feature	Detail
Schema	None — Spark doesn't know the data structure
Optimization	No automatic optimization (Catalyst not used)
Data types	Works with any type: structured, semi-structured, unstructured
API style	Functional — `map`, `filter`, `flatMap`

Three key properties:

```
1. RESILIENT   — if a partition is lost, Spark recomputes it from lineage
2. DISTRIBUTED — split into partitions processed in parallel across executors
3. IMMUTABLE   — transformations always produce a new RDD, never modify in place
```

Transformations are **lazy** (they build a DAG but do nothing); **actions** trigger execution. RDDs are best for unstructured data and custom low-level algorithms — for structured data, prefer DataFrames.

> **Hands-on practice → Notebook 2: RDDs, Transformations & Actions**

14.2 — DataFrame

Introduced in Spark 1.3 — a **distributed collection of data organized into named columns**, like a table in a relational database.

Feature	Detail
Schema	Schema-aware (named columns + data types)
Optimization	Catalyst Optimizer + Tungsten Execution Engine
API style	Declarative — `select`, `filter`, `groupBy`, `agg`
Language support	Python, Scala, Java, R

DataFrames are the right choice for 90% of modern Spark workloads — ETL, analytics, BI, and ML preprocessing. RDDs are a low-level legacy API; only reach for them when DataFrames genuinely can't solve your problem.

> **Hands-on practice → Notebook 3: DataFrames & Spark SQL**

14.3 — Dataset

Introduced in Spark 1.6 — combines type safety (like RDDs) with optimized execution (like DataFrames). Available in **Scala and Java only**. Python users work exclusively with DataFrames, which behave equivalently.

14.4 — Comparison

Feature	RDD	DataFrame	Dataset (Scala/Java)
Schema	None	Named columns + types	Named columns + compile-time types
Optimization	None	Catalyst + Tungsten	Catalyst + Tungsten
Language support	Python, Scala, Java	Python, Scala, Java, R	Scala, Java only

Feature	RDD	DataFrame	Dataset (Scala/Java)
Performance	Slower	Faster	Faster + type safety
Best for	Unstructured / raw data	Structured / semi-structured	Type-safe structured data
API style	Functional	Declarative	Both

15. Spark SQL & DataFrames in Practice

Spark offers two equivalent ways to query structured data — the **DataFrame API** and **Spark SQL**. Both compile to the same execution plan via the Catalyst Optimizer.

How Catalyst Optimizes Queries

```
Your SQL / DataFrame Code
         │
         ▼
  Unresolved Logical Plan
         │  (Analysis — resolve column names, types)
         ▼
  Resolved Logical Plan
         │  (Optimization — predicate pushdown, column pruning)
         ▼
  Optimized Logical Plan
         │  (Physical Planning — choose join strategies)
         ▼
  Physical Plan → Code Generation (Tungsten)
         │
         ▼
       RESULT
```

Key operations covered in the notebook include `select`, `filter`, `withColumn`, `groupBy/agg`, `join`, window functions, handling missing data, and reading/writing CSV, JSON, and Parquet. Parquet is the recommended format for analytics — columnar, compressed, and supports predicate pushdown.

> **Hands-on practice → Notebook 3: DataFrames & Spark SQL**

16. Machine Learning with MLlib

MLlib is Spark's scalable machine learning library — the same code runs on a laptop for development and across a cluster in production.

Algorithm Families

Category	Algorithms
Regression	LinearRegression, DecisionTreeRegressor, RandomForestRegressor, GBTRegressor

Category	Algorithms
Classification	LogisticRegression, RandomForestClassifier, GBTClassifier, NaiveBayes
Clustering	KMeans, BisectingKMeans, GaussianMixture
Utilities	VectorAssembler, StringIndexer, OneHotEncoder, StandardScaler, PCA

The Pipeline Pattern

A `Pipeline` chains preprocessing and modelling steps into one reusable, leak-free object. All fitting happens only on training data, and the same fitted transformations are applied consistently to test and production data.

```
StringIndexer → OneHotEncoder → VectorAssembler → StandardScaler → Model
                                                                    |
                                                          Predictions +
Evaluation
```

`CrossValidator` wraps the entire pipeline for automated hyperparameter search, guaranteeing correct train/test separation at every fold.

> **Hands-on practice → Notebook 4: Machine Learning with MLlib**

---

17. Structured Streaming

Process **live data streams** using the same DataFrame/SQL API as batch — no new API to learn.

Core Concept

```
┌──────────────────────────────────────────────────────────────┐
│  Key Idea: treat a stream as an UNBOUNDED TABLE               │
│                                                               │
│  Batch:   [fixed table]  → process once → result              │
│  Stream:  [table grows continuously] → process incrementally  │
└──────────────────────────────────────────────────────────────┘
```

Architecture

```
DATA SOURCES              SPARK ENGINE              OUTPUT SINKS
────────────              ────────────              ────────────
 Kafka          ──►       Windows          ──►      Console
 Files                    Aggregations              Parquet files
 Rate API                 Joins                     Database
 Socket                   Watermarks                Kafka
```

Output Modes

Mode	Behavior	Best For
Append	Only new rows written each trigger	Filters, transformations
Complete	Full result table rewritten	Running aggregations
Update	Only changed rows written	Stateful aggregations

Window Operations

```
TUMBLING WINDOW (non-overlapping, fixed size)
─────────────────────────────────────────────
 [  10s  ] [  10s  ] [  10s  ] [  10s  ]
 0        10        20        30        40  ← time (seconds)

SLIDING WINDOW (overlapping)
────────────────────────────────────────────
 [    20s    ]
       [    20s    ]
             [    20s    ]
 0    5    10    15    20    25    30       ← time (seconds, slide=5s)
```

Watermarking handles late-arriving data by defining how long Spark waits before dropping events older than a threshold relative to the current stream time.

> **Hands-on practice → Notebook 5: Spark Structured Streaming**

---

18. Performance Optimization

Understanding Spark Execution

```
Job (one action call)
  └── Stage 1 (narrow transformations — no shuffle)
  │     └── Task 1 … Task N  (one per partition)
  └── Stage 2 (after shuffle — wide transformation)
        └── Task 1 … Task N
```

Narrow vs Wide Transformations

```
NARROW (no shuffle — fast)       WIDE (shuffle — expensive)
──────────────────────────       ──────────────────────────
map()                            groupBy()
filter()                         join()
flatMap()                        distinct()
union()                          repartition()
```

```
P1 → P1'                          P1 ─┐
P2 → P2'     (1-to-1)             P2 ─┼─► [Shuffle] ─► P1'
P3 → P3'                          P3 ─┘                P2'
```

Performance Cheat Sheet

Problem	Solution
Same DataFrame used 3+ times	`df.cache()`
Slow join with small lookup table	`broadcast(small_df)`
Too many shuffle partitions	`spark.sql.shuffle.partitions = N` (rule: 2× total cores)
Slow reads from CSV	Convert to Parquet
Python UDF bottleneck	Replace with `pyspark.sql.functions` builtins
Large DataFrame, filter applied late	Move `filter()` earlier in the chain
`collect()` on huge dataset	Use `show()`, `take()`, or write to file

> **Hands-on practice → Notebook 6: Performance Optimization**

---

19. Delta Lake & the Lakehouse

The Problem with Plain Parquet

Parquet is fast and columnar, but it has no built-in concept of transactions. Writing to a Parquet table is not atomic — a failed job can leave partial files, a concurrent reader can see half-written data, and there is no way to "undo" a bad write without manually deleting files.

What Delta Lake Adds

Delta Lake is an open-source storage layer that sits on top of Parquet and adds a transaction log (the `_delta_log/` directory). Every read and write goes through that log, giving you:

Feature	What it means in practice
ACID transactions	Writes are all-or-nothing; concurrent readers always see a consistent snapshot
Time travel	Query any previous version of the table with `VERSION AS OF N` or `TIMESTAMP AS OF '2025-01-01'`
Schema enforcement	Delta rejects writes whose schema doesn't match the table — no silent corruption
Schema evolution	Opt-in column additions (`mergeSchema`) without rewriting the whole table
Upserts (`MERGE`)	Update-or-insert in one atomic operation — essential for CDC pipelines
Z-ordering	Cluster data by high-cardinality columns to minimize files scanned per query

The Lakehouse Pattern

```
Traditional separation                Lakehouse (Delta Lake)
──────────────────────                ──────────────────────────────

 Data Lake  (cheap, raw, no ACID)      One storage layer
     +                            ──►  Parquet files + _delta_log/
 Data Warehouse (ACID, expensive)      ACID + BI + ML — all from the same table
```

This is the architecture behind **Databricks**, **Azure Synapse Delta**, and **AWS Lake Formation** with Delta. In industry, "Delta table" is now as common a term as "Parquet file."

PySpark + Delta Lake (quick look)

```python
# Write a Delta table (same API as Parquet, different format)
df.write.format("delta").mode("overwrite").save("/tmp/apartments_delta")

# Read it back
df_delta = spark.read.format("delta").load("/tmp/apartments_delta")

# Time travel — read the version before the last overwrite
df_v0 = spark.read.format("delta").option("versionAsOf",
0).load("/tmp/apartments_delta")

# Upsert (MERGE) — update existing rows, insert new ones
from delta.tables import DeltaTable
target = DeltaTable.forPath(spark, "/tmp/apartments_delta")
target.alias("t").merge(
    new_data.alias("s"),
    "t.City = s.City AND t.Square_Area = s.Square_Area"
).whenMatchedUpdateAll().whenNotMatchedInsertAll().execute()
```

> **In this module** we work with plain Parquet — Delta Lake is introduced in Module 6 (Databricks). This section is here so the terminology is not surprising when you encounter it.

---

20. Summary & Next Steps

Component Summary

Component	What It Does	When To Use
RDD	Low-level distributed collection	Custom algorithms, unstructured data
DataFrame	Structured data with schema	ETL, analytics, BI, ML preprocessing
Spark SQL	SQL interface over DataFrames	Ad-hoc queries, reporting, dashboards
MLlib	Scalable ML algorithm library	Regression, classification, clustering

Component	What It Does	When To Use
Structured Streaming	Real-time data processing	IoT, clickstreams, fraud detection
Performance Tools	Caching, partitioning, broadcast	Production workloads, large datasets

When to Use Spark vs Pandas

```
Data Size:    < 1 GB          1 GB – 100 GB        > 100 GB
              ──────────────  ────────────────────  ──────────────
Use:          pandas          Either                Spark only
              (simpler)       (benchmark both)      (distributed)
```

Next Steps

1. **Work through all 5 Google Colab notebooks** — hands-on practice is essential
2. **Explore Databricks** — managed Spark + Delta Lake environment; see Section 19 for a Delta Lake primer, full coverage in Module 6
3. **Read:** *Spark: The Definitive Guide* — Chambers & Zaharia (O'Reilly)
4. **Practice:** Try the Spark UI at `localhost:4040` while running jobs — understand stages and tasks
5. **Explore:** Apache Kafka for production streaming sources with Structured Streaming

---

307401 — Big Data and Data Warehouses | Module 5 — Introduction to Apache Spark